

System design собес: выбор базы данных

После требований и оценки нагрузки идёт вопрос, где хранить данные. Задача не угадать «правильную» базу, её нет, а связать паттерн доступа с устройством хранилища и обосновать выбор. Поэтому начинаем с того, как БД устроена внутри: устройство и задаёт, что она делает хорошо, а что плохо.

Движок: B-tree и LSM-дерево

Почти всё поведение базы (быстро ли пишет, быстро ли читает, тянет ли поток записей) определяется тем, на какой структуре данных она хранит записи на диске. Для собеседа достаточно различать две, и они зеркальны.

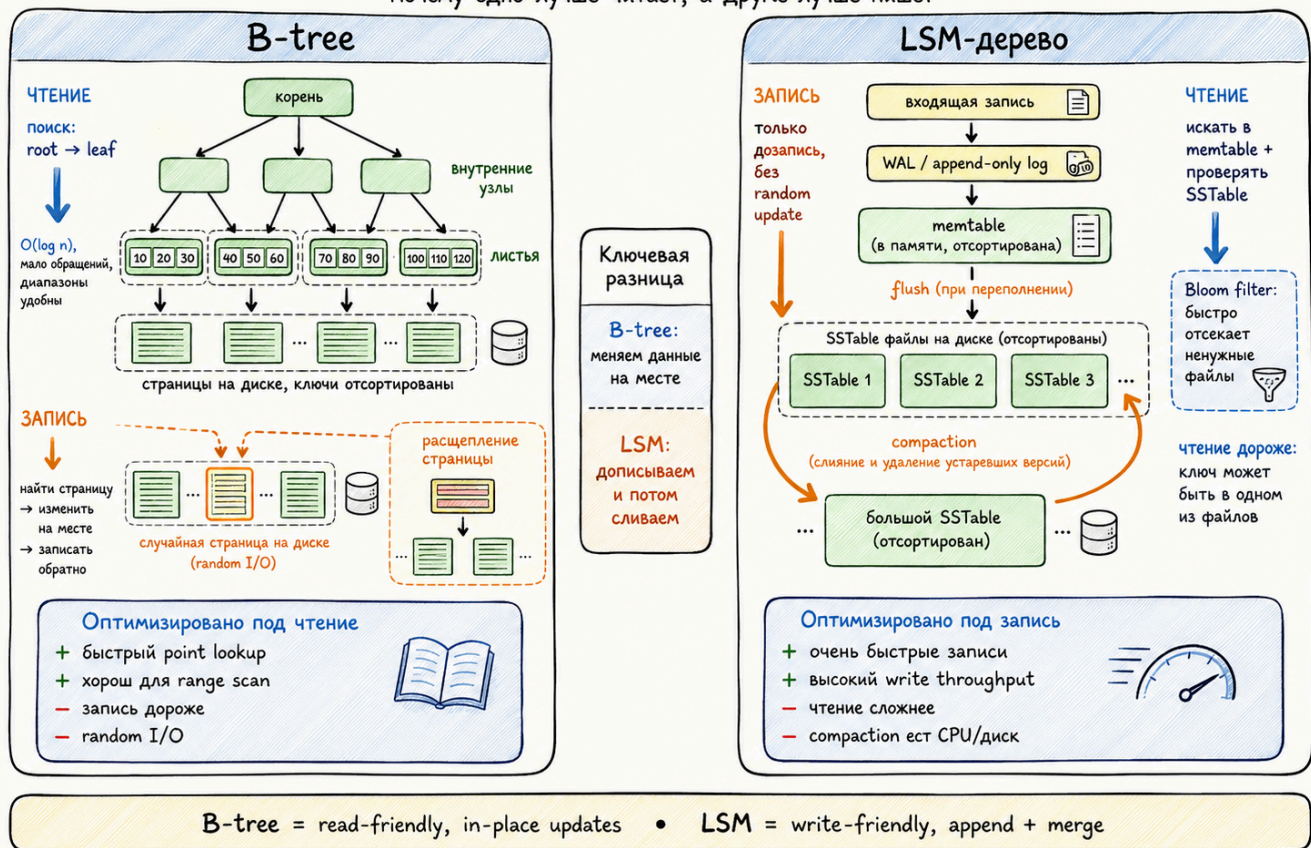
B-tree (B-дерево, на практике B+tree; не путать с бинарным). Основная структура индексов почти во всех реляционных БД. Ключи лежат отсортированными по страницам (блокам фиксированного размера), поиск спускается от корня к листу за $O(\log n)$ и пару обращений к диску, поэтому чтение точечное и по диапазону дёшево. Запись идёт «на месте»: найди страницу, измени, запиши обратно. Это случайный доступ по диску (прыгать по разным местам медленнее, чем писать подряд), а при переполнении страница расщепляется надвое. Итог: читает быстро, пишет дороже. Так работают индексы в PostgreSQL и MySQL (в MySQL/InnoDB сама таблица кластеризована B+tree по первичному ключу, то есть строки лежат внутри индекса; в Postgres таблица это heap, неупорядоченный файл строк, а B-tree поверх неё отдельный индекс) и движок WiredTiger в MongoDB (по умолчанию B-tree).

LSM-дерево (Log-Structured Merge). Перевернутая идея: то, что уже на диске, не трогаем, только дописываем. Новая запись попадает в память (memtable, отсортированная структура) и в append-only лог на диске (на случай сбоя). Когда memtable наполняется, его целиком сбрасывают на диск отдельным неизменяемым файлом (SSTable). Файлы периодически сливаются в фоне (compaction), выкидывая старые версии. Запись это последовательные дозаписи, обычно без чтения-перед-записью, поэтому очень быстрая и держит огромный поток. Плата на чтении: значение может лежать в любом из нескольких SSTable, приходится проверять несколько (помогают bloom-фильтры, они дёшево отсекают файлы, где ключа точно нет), плюс compaction тратит CPU и диск. Итог: LSM быстрее пишет, дороже читает. На нём стоят Cassandra, ScyllaDB, HBase.

Один вывод стоит держать в голове: **Cassandra пишет молниеносно** (LSM плюс отсутствие одного мастера на запись, нагрузка размазана по узлам). Это операционное хранилище под поток записей, а не аналитическое.

B-tree vs LSM-дерево

Почему одно лучше читает, а другое лучше пишет



Движок задаёт компромисс: B-tree быстрее читает, LSM быстрее пишет.

Раскладка на диске: строки и колонки

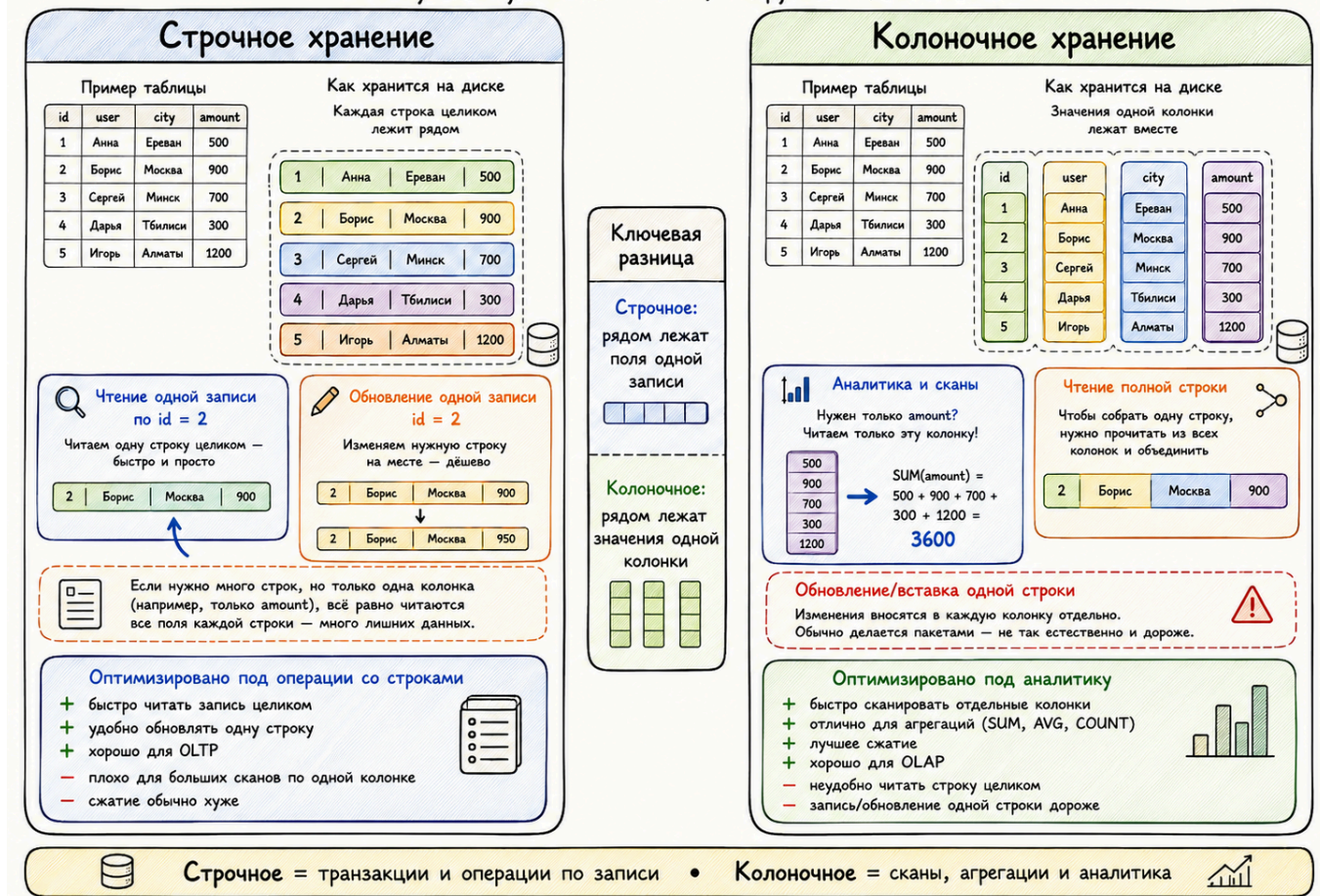
Это вторая, независимая ось: движок (как пишем и читаем) и раскладка (в каком порядке поля строки лежат на диске) сочетаются независимо.

Строчное хранение. Все поля одной записи лежат рядом: id, имя, город, сумма одной строки идут вплотную. Прочитать или обновить целую строку дёшево. Это про операционную работу (OLTP): взять заказ, обновить профиль, провести платёж. Так хранят реляционные БД.

Колоночное хранение. Раскладка перевёрнута: рядом лежат все значения одной колонки по всем строкам (все id вместе, все сумма вместе). Зачем: если запрос трогает одну-две колонки из двадцати (например «средняя сумма по всем заказам»), строчная БД обычно поднимает с диска строки целиком, со всеми колонками, а колоночная читает только нужную колонку. Плюс соседние значения это один тип, они похожи и жмутся в разы. Это про аналитику (OLAP): сканы и агрегации по миллиардам строк. Минус зеркальный: собрать строку целиком или обновить её дорого, правки обычно идут пачками. Так хранят ClickHouse, Vertica, DuckDB.

Строчное хранение vs Колоночное хранение

Почему одно удобно для OLTP, а другое — для аналитики



Строчное хранение под операции со строками (OLTP), колоночное под сканы и агрегации (OLAP).

Wide-column: ещё одна «колонка»

Тут начинается путаница, ради которой и написан этот раздел. Есть третья вещь, в названии которой тоже сидит слово «колонка», и она не имеет отношения к колоночному хранению.

Wide-column (она же column-family) это не про раскладку на диске, а про модель данных. У записи два уровня ключа: **partition key** определяет, на каком узле лежат данные, а **clustering key** задаёт порядок строк внутри партии. То есть под одним partition key лежит много строк, отсортированных по clustering key. Пример: partition key это chat_id, а строки внутри это сообщения чата, отсортированные по timestamp. Данные одной партии лежат на диске рядом и пишутся на LSM-дерево, поэтому доступ по ключу партии и диапазон по времени дешёвы. Так устроены Cassandra, ScyllaDB, HBase, Bigtable. Старое название «column-family», слово «wide» (широкая строка) и образ «миллионы колонок» идут от Bigtable; в современной Cassandra это строки с clustering key, а раздувать одну партию до гигантской вредно.

Разница в одной фразе:

- **Колоночное хранение (columnar)** это «как байты лежат на диске» (по колонкам), ради аналитики и сканов. ClickHouse.
- **Wide-column** это «какая модель данных» (partition key плюс строки по clustering key), ради быстрой записи и доступа по ключу. Cassandra.

	Wide-column (Cassandra, HBase, Scylla)	Колоночное / columnar (ClickHouse, Vertica)
Что значит «колонка»	partition key плюс строки по clustering key	значения колонки физически лежат вместе
Хранение	LSM, строки по ключу партиции	колоночная раскладка плюс сжатие
Сильна в	записи и доступе по ключу	сканах и агрегациях по колонкам
Запись	очень быстрая, огромный поток	пачками, одиночная строка дорого
Назначение	операционка на масштабе (OLTP)	аналитика (OLAP)

Cassandra и ClickHouse это разные базы под разные задачи, и единственное, что их роднит, это слово «column» в описании.

Wide-column vs Columnar

Почему Cassandra и ClickHouse — это разные "колоночные" базы

Wide-column (Cassandra, Scylla)

partition key = chat_id

chat_id = 42

10:01 → Привет
10:02 → Хай
10:03 → Как дела?
...

рядом лежат
данные одного
ключа

Хорошо для:

- + поток записей
- + чтение по ключу
- + история сообщений / события

Типичный запрос:

дай последние 50 сообщений чата 42

Это OLTP / operational storage

Главная разница

Wide-column = модель данных

Columnar = раскладка на диске

Wide-column: данные одного ключа •
Columnar: значения одной колонки

Columnar (ClickHouse)

chat_id	time	user	amount
42	10:01	alice	120
42	10:02	bob	230
7	10:03	alice	150
42	10:04	carol	80

chat_id	time	user	amount
42	10:01	alice	120
42	10:02	bob	230
7	10:03	alice	150
42	10:04	carol	80
...	...	...	...

рядом лежат
значения одной
колонки

Хорошо для:

- + сканы
- + агрегации
- + отчёты / аналитика

Типичный запрос:

посчитай SUM(amount) по всем строкам

Это OLAP / analytical storage

Cassandra: доступ по partition key и быстрая запись • ClickHouse: сканы и агрегации по большим данным

Слева модель данных (partition key плюс clustering key, Cassandra), справа раскладка на диске (по колонкам, ClickHouse).

Гарантии: ACID, BASE, CAP

ACID это четыре гарантии, которые даёт транзакция в реляционной БД:

- **Atomicity** (атомарность): транзакция выполняется целиком или никак, промежуточных состояний не остаётся.
- **Consistency** (согласованность): после транзакции инварианты и ограничения базы целы (баланс не ушёл в минус, внешние ключи валидны).
- **Isolation** (изолированность): параллельные транзакции не видят промежуточных результатов друг друга.
- **Durability** (надёжность): подтверждённое переживёт сбой и перезагрузку.

BASE это мягкий подход NoSQL, контраст к ACID: **B**asically **A**vailable (в основном доступна), **S**oft state (состояние может быть временно нестрогим), **E**ventual consistency (реплики сходятся не сразу, а со временем). Отдаём строгие гарантии ради доступности и масштаба.

CAP это про распределённую БД, где данные лежат в нескольких репликах (копиях на разных узлах). Три свойства:

- **Consistency** (согласованность): любое чтение видит последнюю запись, как будто копия одна (формально это линеаризуемость, linearizability). Это другое C, не из ACID: тут про согласие реплик между собой, там про инварианты внутри одной транзакции.
- **Availability** (доступность): каждый живой узел отвечает на запрос.
- **Partition tolerance** (устойчивость к разделу): система спроектирована пережить потерю связи между узлами.

Теорема говорит: когда сеть распалась на части (раздел), одновременно держать согласованность и доступность нельзя, приходится выбрать одно.

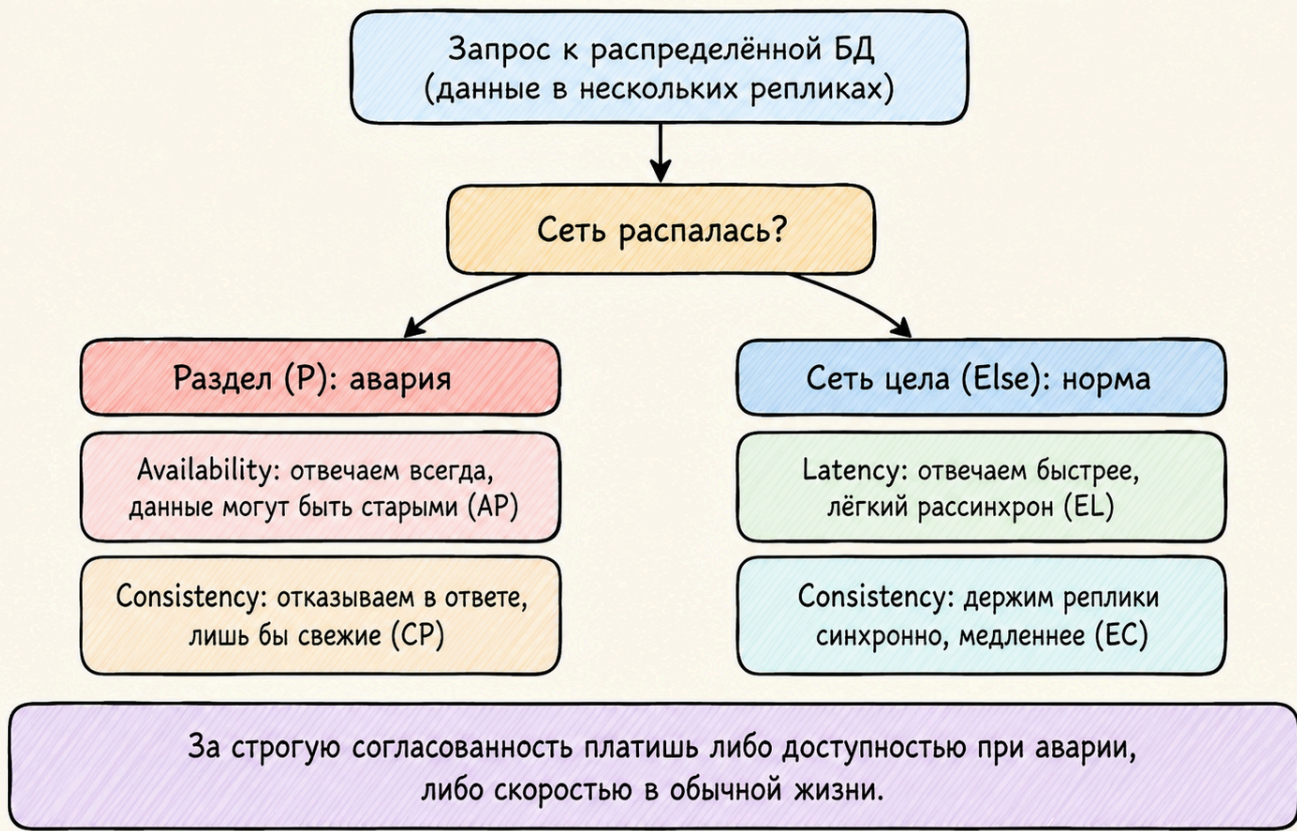
- **CP**: бережём согласованность. На время раздела часть узлов отказывает в ответе, лучше ошибка, чем устаревшие данные. Так обычно делает платёжное ядро (ledger).
- **AP**: бережём доступность. Отвечаем всегда, но можем отдать устаревшие данные, пока реплики не сошлись. Так может позволить себе лента, лайки, счётчики.

Раздел сети в реальности неизбежен (узлы и каналы падают), поэтому на практике выбор сводится к этим двум, CP или AP. Уровень согласованности у многих баз настраивается, так что жёстко клеить продукту ярлык CP или AP не стоит, смотри на конкретную конфигурацию.

PACELC дополняет CAP тем, о чём тот молчит: компромисс есть и когда сеть в порядке. Читается так: если случился раздел (**P**), выбираешь между доступностью и согласованностью (как в CAP); иначе, когда всё хорошо (**E**lse), всё равно выбираешь между задержкой (**L**atency) и согласованностью, потому что держать реплики строго синхронными это лишние обращения по сети туда-обратно и потому медленнее. Короче: за строгую согласованность всегда платишь либо доступностью при аварии, либо скоростью в обычной жизни. Профиль базы и записывают парой букв «что при разделе (**P**) / что в норме (**E**)»: Cassandra это PA/EL (при разделе доступность, в норме низкая задержка), строгая SQL-БД это PC/EC (согласованность в обоих случаях).

PACELC: компромисс есть всегда.

За строгую согласованность всегда чем-то платишь.



Компромисс есть и при аварии (доступность или согласованность), и в норме (задержка или согласованность).

Семейства БД

Каждое семейство по одной схеме: как работает, плюсы, минусы, когда брать.

Реляционные (RDBMS).

Как работает. Данные в таблицах с фиксированной схемой, строки связаны внешними ключами, доступ через SQL с JOIN-ами (соединять таблицы по ключу в одном запросе). Всё через транзакции с гарантиями ACID, индексы на B-tree.

Плюсы:

- Целостность из коробки: транзакции, ограничения и внешние ключи не дают данным разъехаться.
- Гибкие запросы, которые не знаешь заранее: ad-hoc SQL (любой запрос на ходу) и JOIN-ы.
- Зрелость: инструменты, документация, понятная всем модель.

Минусы:

- Запись идёт через один узел-мастер (классическая primary-replica схема), и масштаб упирается в его потолок (про обход ниже).
- Схема жёсткая: поменять структуру это миграция.

Когда брать. Почти всегда как старт, особенно где есть деньги, заказы, права доступа, связи между сущностями.

Представители. PostgreSQL, MySQL. По умолчанию Postgres: он умеет ещё и документы (JSONB, бинарный JSON прямо в колонке), гео (PostGIS), векторы (pgvector), временные ряды (TimescaleDB), то есть закрывает многое без отдельной БД.

Когда реляционки мало по масштабу: шардинг и distributed SQL.

Реляционная БД в первую очередь растёт вертикально: берёшь машину мощнее. Чтение можно разгрузить репликами, но запись всё равно идёт через один мастер, и у этого есть потолок. Дальше два пути расширить её на много машин.

Шардинг (sharding). Ты сам режешь таблицу на куски (шарды) по ключу, например по `user_id`, и раскладываешь шарды по разным серверам, каждый держит свою часть данных. Маршрутизатор по ключу решает, на какой шард идёт запрос. Это даёт горизонтальный масштаб, но цена высокая: запрос или транзакция, которым нужны данные с разных шардов (JOIN между пользователями, перевод денег между шардами), становятся сложными или невозможными, ключ шардирования надо выбрать заранее и удачно, а перебалансировка болезненна. По сути ты руками приделываешь масштаб к базе, которая для этого не задумывалась (помогает middleware вроде Vitess: прослойка-маршрутизатор поверх БД).

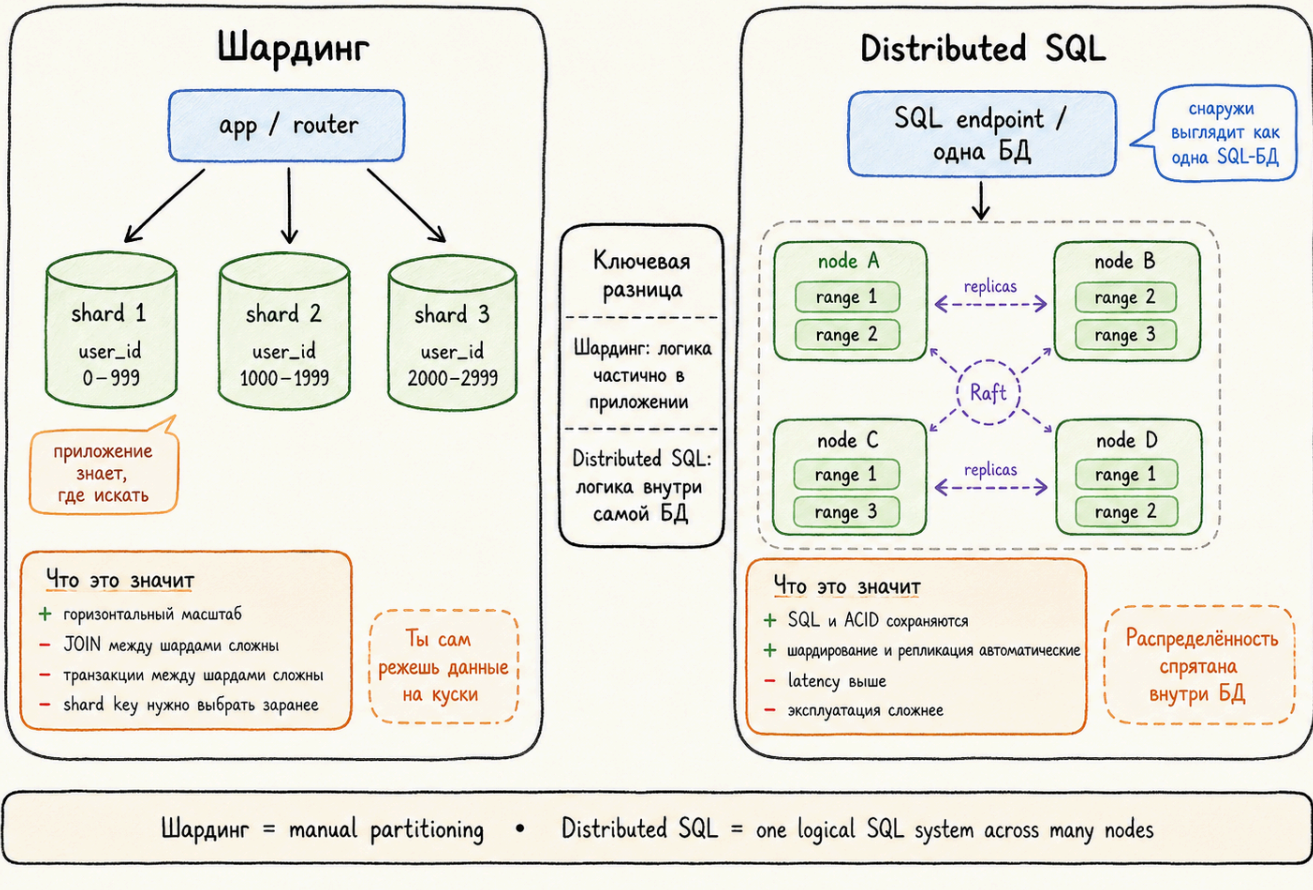
Distributed SQL (NewSQL). База, изначально построенная как единая SQL-система, размазанная по многим узлам: она сама шардирует и реплицирует данные, но снаружи выглядит как одна обычная SQL-БД и сохраняет транзакции и JOIN-ы, в том числе между узлами. Внутри данные автоматически бьются на диапазоны по узлам, для согласованности и репликации каждый диапазон гоняет консенсус (Raft/Paxos, узлы голосуют за общий порядок), а транзакции между узлами держит специальный протокол (у Google Spanner для этого синхронизированные часы TrueTime). Платишь повышенной задержкой на запросах и транзакциях между узлами (locality, держать связанные данные на одном узле, всё равно важна) и сложностью эксплуатации.

Разница в одной фразе. Шардинг это масштаб поверх одиночной БД ценой потери кросс-шардовых JOIN-ов и транзакций; distributed SQL это база, которая распределённая по своей природе и сохраняет SQL и ACID за тебя (ценой задержки на кросс-узловых операциях).

Представители distributed SQL. Google Spanner, CockroachDB, YugabyteDB, TiDB.

Шардинг vs Distributed SQL

Ручное масштабирование против распределённой SQL-БД



Шардинг это ручное разрезание поверх одиночной БД; distributed SQL распределён по природе и сохраняет SQL и ACID за тебя (кросс-узловые операции медленнее).

Key-value (ключ-значение).

Как работает. Модель доступа ключ → значение: кладёшь и достаёшь по ключу, без запросов по значению и JOIN-ов (внутри это хеш-таблица, B-tree или LSM, но снаружи просто «дай по ключу»). Часто целиком в оперативной памяти.

Плюсы:

- Очень низкая задержка и простота.
- Легко масштабировать: ключи равномерно раскидываются по узлам.

Минусы:

- Только доступ по ключу, никакой аналитики и сложных запросов.
- In-memory вариант ограничен объёмом RAM и теряет данные при перезапуске без персистентности (сохранения на диск).

Когда брать. Кэш, сессии, счётчики, rate-limit (ограничение частоты запросов), лидерборды, очереди. Чаще это слой кэша перед долговечной базой, реже самостоятельное хранилище.

Представители. Redis, Valkey (открытый форк Redis), Memcached. DynamoDB тоже

относят сюда: это управляемый key-value с моделью partition key плюс sort key (wide-row стиль).

Wide-column (column-family).

Как работает. Partition key (где лежит) плюс clustering key (порядок внутри), на LSM-дереве (модель разобрали выше). Партиции раскиданы по узлам по хешу ключа, внутри партиции строки отсортированы по clustering key, часто по времени.

Плюсы:

- Огромный поток записей: LSM пишет последовательно.
- Линейный горизонтальный масштаб: добавил узлы, выросла пропускная.
- Быстрый доступ по ключу партиции и по диапазону clustering key.

Минусы:

- Нет JOIN-ов и произвольных запросов: таблицу проектируешь заранее под конкретный запрос и денормализуешь (дублируешь данные под каждый запрос вместо соединений).
- По умолчанию eventual consistency: реплики сходятся не мгновенно, какое-то время можно прочитать старое значение.

Когда брать. Гигантский поток по ключу, который можно денормализовать: лента, сообщения, события, телеметрия.

Представители. Cassandra, ScyllaDB (переписана на C++, быстрее), HBase, Google Bigtable.

Колоночные / аналитические (columnar).

Как работает. Колоночное хранение на диске (разобрали выше): значения колонки лежат вместе, запрос читает только нужные колонки, данные сильно жмутся. Запись обычно пачками.

Плюсы:

- Быстрые сканы и агрегации (SUM, AVG, COUNT) по немногим колонкам над миллиардами строк.
- Сильное сжатие за счёт однотипных соседних значений.

Минусы:

- Точечный доступ и обновление одной строки дорогие (правки идут пачками): строку надо собирать из всех колонок.
- Запись пачками, не для потока одиночных записей.

Когда брать. Аналитика, отчёты, дашборды, сбор фич для ML из логов (OLAP).

Представители. ClickHouse, DuckDB (встраиваемый, «SQLite для аналитики»), Snowflake и BigQuery (облачные склады), Vertica.

Документные.

Как работает. Хранит документы (часто JSON-подобные; MongoDB внутри BSON) с гибкой вложенной схемой: у разных документов могут быть разные поля. Запрос по полям и вторичным индексам, частичная агрегация. Под капотом обычно B-tree.

Плюсы:

- Гибкая схема: добавил поле в документ без миграции.
- Естественно ложится, когда сущность это самодостаточный документ с вложенностью.

Минусы:

- Плохо со связями многие-ко-многим и JOIN-ами: приходится денормализовать и ловить рассинхрон.
- Междокументные транзакции есть, но это не её конёк.

Когда брать. Каталоги товаров с разным набором атрибутов, профили, CMS; достаёшь документ по id или паре индексов.

Представители. MongoDB, CouchDB, Amazon DocumentDB. Для умеренных нужд Postgres с JSONB закрывает это без отдельной БД.

Object storage (S3).

Как работает. Это не база, а хранилище объектов: кладёшь файл под ключом и достаёшь его целиком по ключу, обычно по HTTP. Внутри плоское пространство «ключ → байты», без таблиц, схемы, запросов и транзакций. Рассчитано на гигантские объёмы и высокую надёжность (данные лежат в нескольких копиях), масштабируется практически без предела.

Плюсы:

- Дёшево за гигабайт и почти бесконечный объём.
- Высокая надёжность (durability): копии в нескольких местах.
- Простая отдача файлов, удобно поставить перед ним CDN.

Минусы:

- Никаких запросов: только положить, достать или удалить по ключу.
- Не для мелких частых изменений и не для низкой задержки уровня БД.

Когда брать. Большие бинарные файлы: видео, фото, вложения, бэкапы, статика. В базе держишь только метаданные и ссылку на объект, сам файл в S3.

Представители. Amazon S3, Google Cloud Storage, Cloudflare R2, MinIO.

Что ещё бывает. Полнотекстовый поиск это Elasticsearch / OpenSearch. Временные ряды: мониторинг это Prometheus, специализированные time-series БД это InfluxDB и TimescaleDB. Графы (обходы «друзья друзей», антифрод) это Neo4j.

Сводная таблица

Сравнение по главным осям: как пишет, как читает, легко ли разносится по многим машинам (горизонтальный масштаб), какие запросы тянет.

Тип	Запись	Чтение	Гориз. масштаб	Запросы и связи	Лучшее применение
Реляционная (Postgres)	средняя, через один мастер	быстрое, гибкое	тяжело: шардинг руками или distributed SQL	SQL, JOIN, ad-hoc	ядро, деньги, связи
Key-value (Redis)	очень быстрая	очень быстрое по ключу	лёгкий	только по ключу	кэш, сессии, счётчики
Wide-column (Cassandra)	очень быстрая (LSM)	быстрое по ключу и диапазону	лёгкий, линейный	по ключу, без JOIN	поток по ключу: лента, сообщения
Колоночная (ClickHouse)	пачками, строка дорого	очень быстрые сканы и агрегации	лёгкий	аналитический SQL, точечные не профиль	аналитика, OLAP
Документная (Mongo)	быстрая	по полям и индексам	шардинг встроен	по полям, связи слабые	документы переменной формы
Object storage (S3)	положить объект	отдать по ключу (HTTP)	практически без предела	запросов нет	большие файлы

Алгоритм выбора

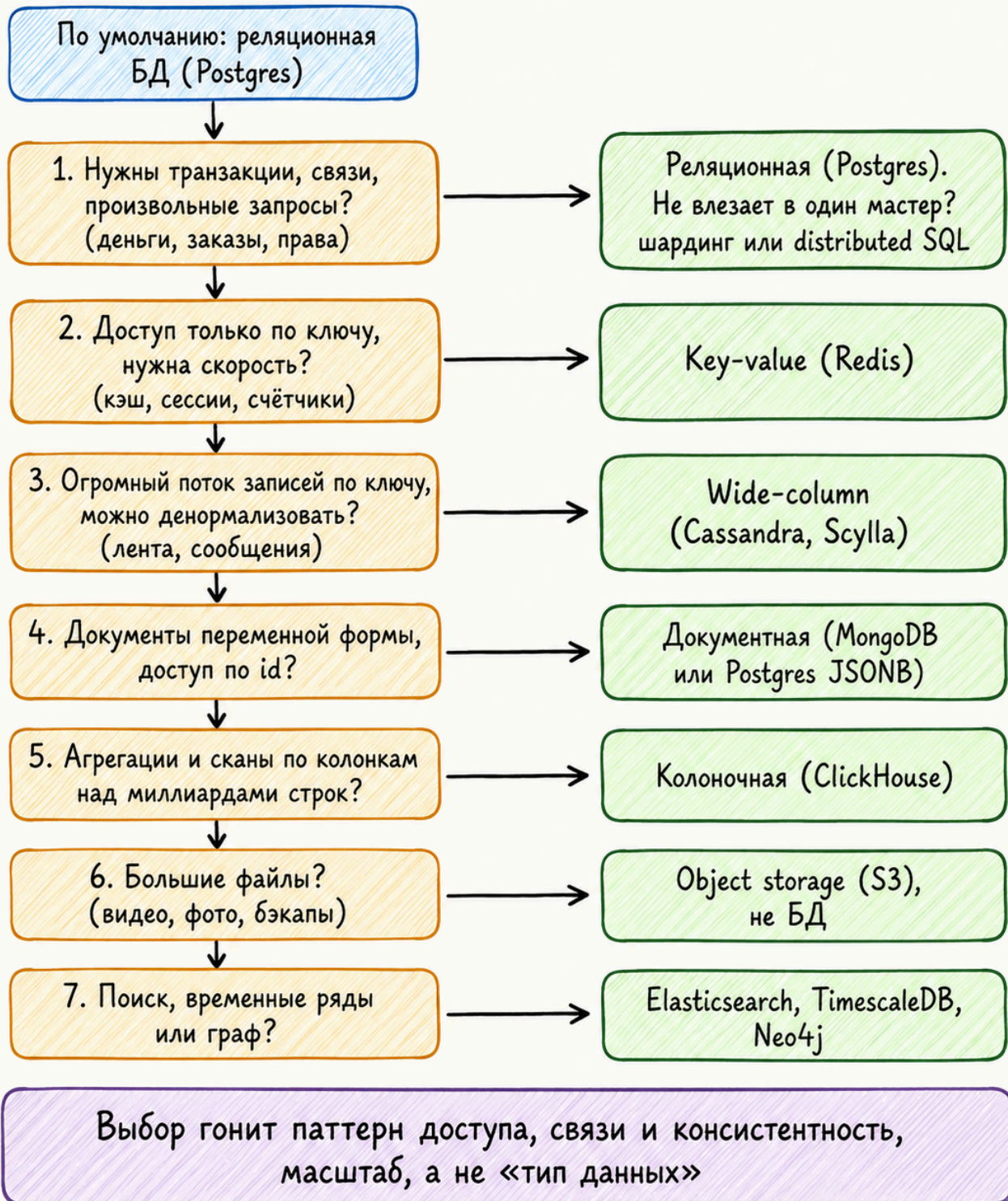
Прогоняй список по каждому классу данных отдельно, не по системе целиком: первое подходящее условие даёт хранилище под этот класс, а система обычно комбинирует несколько.

1. **По умолчанию реляционка (Postgres).** Сначала проверь, не хватит ли её одной; специализированную БД заводи под конкретное узкое место, а не заранее.
2. **Нужны транзакции, инварианты, связи, ad-hoc запросы** (деньги, заказы, права доступа, граф связей)? Реляционная. Не влезает в один мастер по записи или объёму? Шардинг или distributed SQL.
3. **Доступ только по ключу, нужна скорость, счётчики, сессии, кэш?** Key-value (Redis). Чаще как слой кэша перед долговечной базой.

4. **Гигантский поток записей по ключу, который можно денормализовать, eventual ok** (лента, сообщения, события)? Wide-column (Cassandra, Scylla).
5. **Самодостаточные документы переменной формы, достаёшь по id или паре индексов?** Документная (Mongo) или Postgres JSONB.
6. **Агрегации и сканы по немногим колонкам над миллиардами строк, запись пачками?** Колоночная (ClickHouse).
7. **Большие файлы** (видео, фото, бэкапы)? Object storage (S3), в базе только ссылка.
8. **Специальное:** полнотекстовый поиск это Elasticsearch; метрики и временные ряды это time-series; обходы графа это graph.

Выбор базы данных: дерево решений

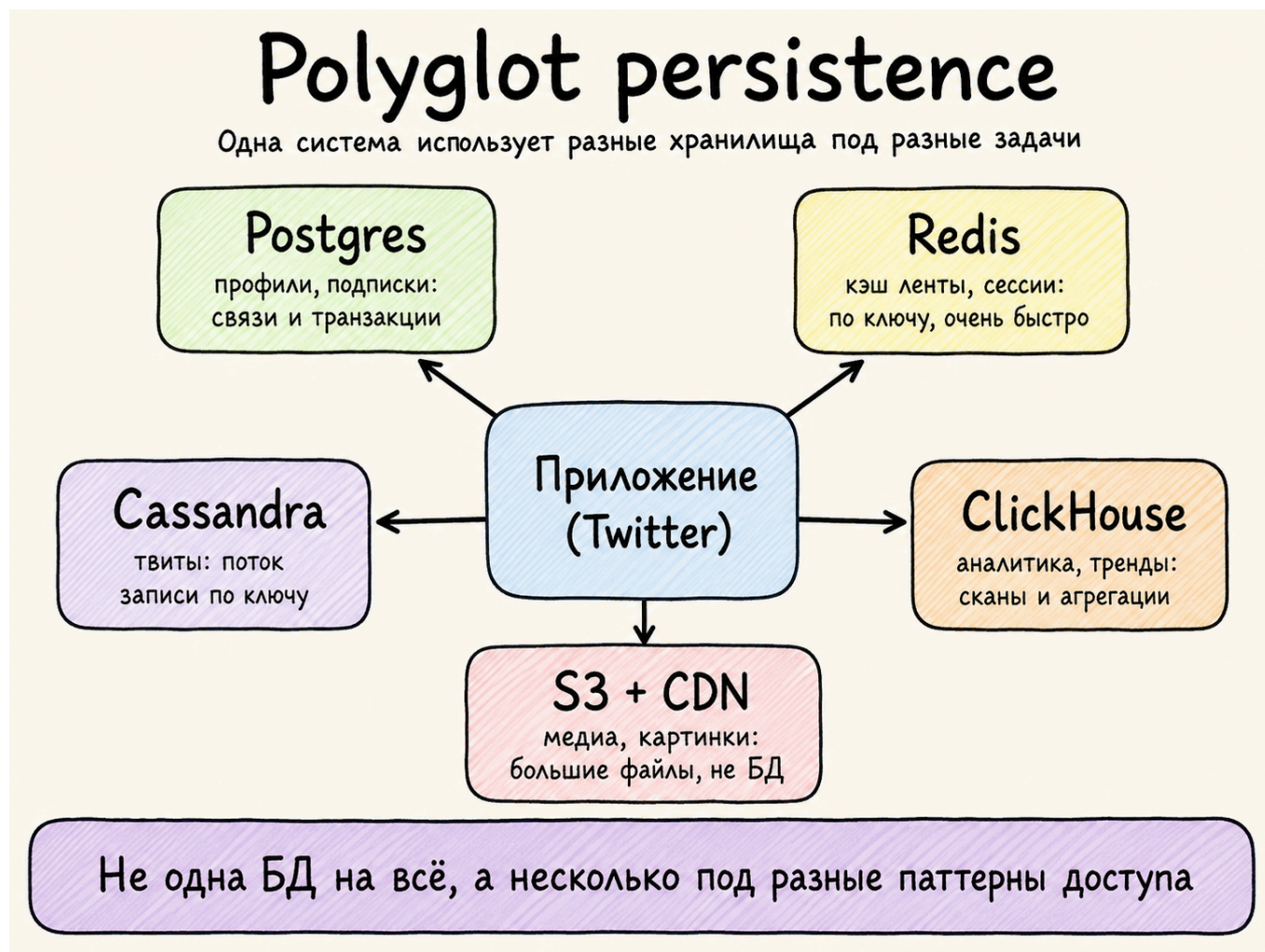
Читаем сверху вниз, на первом подходящем условии сворачиваем по стрелке вправо к ответу



Для каждого класса данных идём сверху вниз; первое подходящее условие даёт хранилище под него.

Выбор гонит не «тип данных», а **паттерн доступа, нужны ли связи и консистентность, и масштаб**. Реальная система обычно использует **несколько баз** под разные части,

каждую под свой паттерн доступа (этот подход называют polyglot persistence, то есть много разных хранилищ в одной системе): Postgres под ядро, Redis под кэш, Cassandra под поток, ClickHouse под аналитику, S3 под файлы.



Одна система держит несколько баз, каждую под свой паттерн доступа.

Примеры

Те же пять систем, что в заметке про нагрузку ([requirements-and-load](#)); числа оттуда. Схема разбора: раскладываем данные системы на части, под каждую подбираем хранилище по её паттерну доступа, и отдельно проговариваем ключевой неочевидный выбор, где есть соблазн взять не то.

☰ Сокращатель ссылок (TinyURL) >

Сервис: длинную ссылку в короткий код, по коду редирект. Денег, связей и транзакций нет; чтений к записям ~100:1, важны задержка редиректа и масштаб чтения.

Данные	Доступ	Хранилище	Почему
код → URL	запись редко, чтение по коду очень много	Postgres (источник истины) + Redis (кэш)	доступ по одному ключу; надёжная база хранит истину, почти все чтения снимает кэш
профиль юзера	по ключу, изредка	Postgres	мало данных, лежит рядом с основной таблицей
клики (аналитика)	агрегации по времени и стране	ClickHouse (или счётчики в Redis)	это сканы и агрегации, работа колоночной БД

Ключевой выбор. Доступ по ключу подмывает взять чистый KV (Redis, DynamoDB) как основное хранилище, но источник истины держим в Postgres: терять ссылки нельзя, а Redis тут только кэш перед ней. Объём при этом небольшой: 100М ссылок/мес × 12 мес × 5 лет = 6 млрд строк × 0.5 КБ = 3 ТБ. Это порядок, который тянет реляционка (при росте реплики и шардинг), а не повод сразу брать Cassandra; специализацию под запись достаёшь, только когда поток реально огромен.

☰ Notion / облачный диск >

Сервис: документы из вложенных блоков, совместное редактирование, права доступа, синхронизация между устройствами. Терять заметки нельзя, права обязаны быть верны; между устройствами хватает eventual.

Данные	Доступ	Хранилище	Почему
метаданные: юзеры, страницы, блоки, права, связи	связи и транзакции, автор сразу видит свою правку (read-your-writes)	Postgres	целостность прав и связи между сущностями это работа реляционки
вложения, картинки, большие блоки	положить и отдать целиком по id	object storage (S3)	большие файлы не место в строке БД, в базе только ссылка
онлайн-состояние совместного редактирования	по ключу, эфемерно	Redis	быстрый доступ по ключу, не жалко потерять при перезапуске

Ключевой выбор. Документ из блоков выглядит как кейс под документную БД (Mongo), но берём Postgres: главное тут не гибкая форма документа, а связи и целостность (страница в воркспейсе, права «кто что видит», ссылки между страницами) плюс транзакции. Файлы выносим в S3, чтобы база не пухла

бинарниками; целостность отдаём её транзакциям, а не дописываем руками поверх хранилища без транзакций.

☰ Лента / Twitter >

Сервис: публиковать твиты, подписываться, видеть домашнюю ленту. Для ленты доступность важнее строгой согласованности (твит секундой позже это норм, AP); аккаунты и платежи, наоборот, строгие. Масштаб записи огромен.

Данные	Доступ	Хранилище	Почему
профили, подписки	связи, чтение	Postgres	связи и целостность; граф подписок как списки связей
твиты	поток записи, чтение по автору и времени	Cassandra / Scylla (wide-column)	partition по автору, sort по времени; денормализация под ленту, LSM держит поток
домашняя лента	чтение по <code>user_id</code> , предвычислена	Redis (KV-кэш)	готовую ленту отдаём по ключу за миллисекунды (fan-out: разложена по подписчикам заранее)
медиа	отдать пользователю	S3 + CDN	большие файлы и дешёвая массовая отдача

Ключевой выбор. Почему твиты в Cassandra, а не в шардированном Postgres: это поток на запись по ключу без JOIN-ов, ровно профиль wide-column (быстрая запись на LSM, линейный масштаб, денормализация под чтение ленты). Саму ленту не собирают запросом на лету, её предвычисляют рассыпкой по подписчикам (fan-out) и держат готовой в кэше. Тренды и «что популярно» это уже аналитика, отдельный колоночный контур (ClickHouse), не Cassandra.

☰ Видеостриминг (Netflix) >

Сервис: смотреть фильмы, искать по каталогу, получать рекомендации. Каталог меняется редко, консистентность не критична; узкое место это полоса отдачи видео, а не БД.

Данные	Доступ	Хранилище	Почему
профили, каталог	связи, чтение	Postgres + Redis	связи; горячий каталог в кэш

Данные	Доступ	Хранилище	Почему
видеофайлы	отдать по сети массово	object storage (S3, geo) + CDN	огромные файлы, БД тут ни при чём
логи просмотров	поток событий, потом агрегации	ClickHouse	аналитика и обучение рекомендаций это сканы по колонкам

Ключевой выбор. Две «колоночные» здесь не путать: логи просмотров идут в columnar ClickHouse (сканы и агрегации над миллиардами событий), а не в wide-column Cassandra, которая про запись по ключу, а не про аналитические сканы. Сами видеофайлы вообще вне БД, в object storage за CDN, потому что узкое место это полоса, а не хранилище или запросы.

☰ Мессенджер (Telegram / WhatsApp) >

Сервис: личные и групповые чаты, статус онлайн, доставка в реальном времени. Порядок сообщений в чате согласован; узкое место это число постоянных соединений.

Данные	Доступ	Хранилище	Почему
онлайн-состояние, соединения	по ключу <code>user_id</code> , эфемерно	Redis	быстро по ключу, не жалко потерять
контакты, профили	связи	Postgres	связи и целостность
история сообщений	поток записи, чтение по чату и времени	Cassandra / Scylla (wide- column)	partition по <code>chat_id</code> , sort по времени, родной паттерн
недавние сообщения	мгновенная выдача	Redis	кэш горячих сообщений
медиа	отдать	S3	файлы вне БД

Ключевой выбор. История сообщений ложится на wide-column идеально: partition key это `chat_id` (все сообщения чата на одном узле, рядом), clustering key это время (читаешь последние N или диапазон одним запросом), а поток записи держит LSM. Онлайн-состояние отдельно в KV, потому что оно эфемерное и нужно мгновенно по ключу. Так Discord и хранит сообщения (это про сообщения, не про всю его архитектуру): сначала Cassandra, потом ScyllaDB ради скорости.

Что не забыть на собеседе:

- «Колоночная» это перегруженный термин: чаще под ним имеют в виду wide-column (Cassandra, операционка, запись), реже колоночное хранение (ClickHouse, аналитика). Всегда уточняй, что именно.
- Default Postgres это не догма, а стартовая точка: отступай от неё под конкретное требование.
- Продукты и лицензии меняются (Redis и форк Valkey, рост ClickHouse), перед собеседом сверяй текущий ландшафт.

Источники:

- [Designing Data-Intensive Applications](#) (Kleppmann): глава 3 про storage engines (B-tree и LSM), глава 5 про репликацию и консистентность
- [CAP Twelve Years Later](#) (Brewer, 2012)
- [How Discord Stores Trillions of Messages](#)
- [Cassandra data modeling](#): partition key, clustering key, проектирование под запрос
- [CockroachDB architecture](#): как distributed SQL раскладывает данные и держит транзакции